

Zult-

interpreted and compiled

i want programs to be fast like C++

i want to be able to use existing libraries

i want to be able to understand whats going without magic like C++ or js or python

i want to make games, apps, programs, simulations and more

C++ power, without C++ complexity; compile anywhere when performance isnt the priority.

Old-school surface, modern internals

@

Zult –

Basic function declaration and returning.

Functions are declared using 'func' keyword followed by a name, returns, and arguments.

Example: `func add[int](int A,int B);`

`[]` are optional for no-return functions.

Functions can have 2 returns, a Value return, and an Error return.

An Error return must be a Primitive-type.

`!` marks a return as no-discard.

``return`` is used to return from functions.

double returns are NOT tuples.

Examples:

`[]:none, [type]:single, [type:err]:double` return.

`[:err]:` error only return.

`[!type] [!:err] [!type:err] [type:!err] [!type:!err]:` no discard.

Arguments are declare with a type, then name, and optionally a default value.

They are separated by comma`,`.

Code-Examples:

```
return;
return val;
return err; functions the same as ret val;
return val,err;
func name(){ block }
func name[](){ block }
func name[ret](){ block }
func add[int](int a,int b){
    return a+b;
}
func load[int:int](){ return data,err; }
```

Errors:

```
func fn[int:int](){ return 0; }
```

```
func fn[int](){ return 0,0; }
```

@

Zult–

Basic if, else, and switch statements.

if:

condition evaluated once.

branch runs if condition not-zero(!0).

can be followed by else branches.

Examples:

```
if(condition){ block }  
if(condition){ block }else{ block }
```

switch:

only accepts interger type values, int, char, enum, etc.

switch value evaluated once.

patterns checked top-to-bottom.

first matching pattern executes.

can be followed by else branches.

only one branch may execute.

auto breaks at end of pattern block.

patterns separated by comma(,) are OR based.

if no pattern matches, else chain is run.

tailing comma is fine.

all patterns work as specified with both positive and negative numbers.

Patterns:

(N): value(value is N).

(A~B): range(value is A or B or inbetween).

(<N): less(value is less then N)

(>N): greater(value is greater then N)

(<=N): less/equal (value is less or equal to N)

(>=N): greater/equal (value is greater or equal to N)

(*): wild card(works like default in C)

examples:

```
switch(val){  
    (pattern){ block }  
    (pattern,pattern){ block }  
}else{ block }: else runs if no pattern matches
```

@

Zult –

else:

else defines an alternative branch
else can optionally have a condition
conditional else runs only if all previous branches failed
unconditional else acts as fallback branch
else chains evaluated top-to-bottom
only one branch in chain may execute

examples:

```
else(condition){ block }  
else{ block }  
if(condition){ block }else{ block }  
if(condition){ block }else(condition){ block }else{ block }  
switch(val){}else{ block }  
switch(val){}else(condition){ block }else{ block }
```

@

Zult –

Primitive-types & type modifiers

int, uint,
float, double,
i8, i16, i32, i64,
u8, u16, u32, u64,
f32, f64,
bool,
void

void is special type.

signed integer types: i8,i16,i32,i64,int.

unsigned integer types: u8,u16,u32,u64,uint.

floating point types: f32,f64,float,double.

Modifiers:

* pointer
& reference
[N] array(of N values)
const immutable
modifiers are parsed left to right
type modifiers modify the type itself

examples:

 *int: pointer to int.
 &*int: reference to pointer to int.
 [5]int: array of 5 ints.
[3][5]int: 3 arrays of 5 ints.
 &[5]int: reference to array of 5 ints.
 [5]*int: array of 5 pointers to ints.

Zult –

Declarations & Assignments.

declarations start with type, then variable names.
all variables are of the specified type(Modifiers included)
uninitialized variables will be given there default value(int -> 0,
ptr -> null, etc).

Examples:

```
type A;  
type A, B;  
type A = val;  
int var;  
int var = 1;  
*int ptr1,ptr2; both are pointers to ints.  
[type,type] var, var = fn(); multireturn function syntax specific
```

assignments store the right value in the left variable(s)

```
Operators:{  
    = += -= *= %= &= |= ^= <<= >>= /=  
}
```

examples:

```
var = 1;  
var = fn();  
val, err = fnErr(); multi return function  
val = fnErr(); functions error is discarded  
err = ?fnErr(); functions value is discarded  
val,_ = fnErr(); functions error is explicitly discarded  
_,err = fnErr(); functions value is explicitly discarded
```

Zult –

Pointers & Casting.

Pointers:

pointers can be dereferenced with `\*`.
pointers can be reassigned to point to something else.
function pointers must match input args and return types exactly.
return discard ! must also match.
to do pointer math, the pointer must be marked with \$
pointer math will use pointer types size, not bytes.

examples:

\$ptr+=1: adding 1 to ptr.
\$ptr[3]: get 3rd value.
*func[val:err](arg1,arg2) function pointer.
*func[int](int,int) fnPtr = add;

casting converts from one type to another type.

examples:

(type)var: cast var to type.
(i32)var: casts var to type i32.
(*int)var: casts var into a pointer to int.

Zult –

Structs:

Structs are raw container for data, declared with ``struct`` followed by a name.

Structs can have variable declarations with optional default values. Members can be accessed with ``.`` operator for both direct structs and pointer struct members.

The structs name is used to define constructors and the destructor.

Example: `myStruct(args){}` constructor.

Add `~` to the beginning of the struct name to define the destructor.

Example: `~myStruct(){}` destructor.

a destructor can not take arguments.

Unlike destructor, constructors can take arguments, and there can be more then one.

Structs are bitwise copied by default using ``=``.

Overloading the `:=` operator will allow you to custom copy/transfer logic.

Note: overloading `:=` disables bitwise copy ``=``.

a constructor with no arguments runs at declaration if no other constructor does.

A destructor runs at end of struct declarations scope.

If a destructor is defined, bitwise copy is disabled ``=``.

Modifiers:

`packed`: makes struct have no padding.

`align(N)`: makes struct align to N bytes.

Examples:

`struct myStruct{char,int}`: 8 bytes or more.

`packed struct myStruct{char,int}`: 5 bytes.

`align(16) struct myStruct{char,int}`: 16 bytes.

```
struct vec2i{
    int x = 0,y = 0;
    vec2i(int xin,int yin){
        x = xin;
        y = yin;
    }
    ~vec2i(){ // functionally useless in this example.
        x=0;
        y=0;
    }
}@
```


Zult –

Member Functions

Member functions are declared with ``func`` followed by the type ``.`` and the functions name, its returns, and its inputs.

Example: `func vec.normalize[vec]() { code }`

Example: `func float.round[float]() { code }`

``self`` is a pointer to the one calling the member function.

Example: `1.3.round()`, `self` is a pointer to the float 1.3.

``self`` can only be used within the member function.

Member functions are not declared within anything, they are global.

Member functions work for structs, enums, and types.

Member functions can be members of namespaces.

Zult –

Operations & Overloading

operators order by precedence:

[]	(left)
.	(unary, left)
! ~ + -	(unary, right)
* % /	(left)
+ -	(left)
<< >>	(left)
&	(left)
^	(left)
	(left)
< > <= >=	(left)
== !=	(left)
	(left)
&&	(left)
:=	(left)

Binary overloadable operations: == != < > <= >= | ^ & << >> + - * % /
[] :=

Unary overloadable operations: + - ~

Example: operation +[vec](const *vec A, const *vec B){ code }

Example: operation -[vec](const *vec A){ code } unary operation.

Inputs and output do not need to be the same.

Example: operation +[float](const *vec A, const *int B){ code }

The input combination must be unique, as in no other overloads use it.

Note that the assignment operation `=` is not overloadable, or even consider an `operation`.

:= is a special operation intended for more custom assignments that are not bitwise like `=`.

Zult –

Enums

An enum is declared with keyword ``enum`` followed by a type and name.

Example: `enum int Err{Failed,Success}`

An enum can have explicit values set by the user.

Enums do not need to be cast to be assigned to there type.

Example: `int val = Err::Success;`

However you need to cast to the enum to assign it a none enum value.

Example: `Err val = (Err)0;`

If an enum is declared without a type, it will default to ``int``.

When used in switch statement, all values must be checked unless a wild card ``*`` or else statement is present.

Enums have a built in member function ``name`` which returns a string representing the current value.

Example: `Err::Success.name();` returns a pointer to a string that says "Success".

Examples:

```
enum type name{
    name,
    name,
    ...
}
enum u8 Err{
    success,
    failed,
    unknown
}
```

Zult –

Namespaces & Using

Namespaces are wrappers for global things like structs, enums, functions, variables, etc.

Namespaces are declared with keyword ``namespace`` followed by a name.

Example: `namespace name{ code }`

``private`` is used to restrict something to only its namespace.

Namespaces can be nested.

Namespace members can be accessed with ``::``.

Example: `namespace::member;`

Namespaces without a name are scoped to the file there in.

Using is used to make aliases.

Example: `using Byte = u8;`

Example: `using String = *char;`

Example: `using Math::PI; imports Math::PI as just PI.`

Example: `using Render = Engine::Graphics::Renderer;`

Zult –

`defer`: delays a function from running until the scope it was called in ends.

To-Do

variadic arguments

Lambdas

Type-Safe Structure-of-Arrays (SoA) Variadic System Specification

Component	Memory Location	Internal Representation	Runtime Cost
Metadata Array (<code>types[]</code>)	<code>.rodata</code> (Static Memory)	Compiler-generated descriptor array	0 allocation cost; passed as a static 16-byte (<code>ptr</code> , <code>len</code>) slice
Data Array (<code>payload[]</code>)	Caller Stack Frame	Contiguous array of uniform 64-bit (8-byte) slots	O(1) indexing; total size = N×8 bytes

Memory & Storage Layout Matrix

Argument Type	Storage Mechanism	Slot Payload	Semantics Preserved
Primitives (≤64-bit)	Direct Slot	Padded raw value	Pass-by-value
Floats (<code>float</code>, <code>double</code>)	Direct Slot	IEEE-754 bit representation	Pass-by-value
Pointers / Strings (<code>char*</code>)	Direct Slot	64-bit memory address	Pass-by-reference
Large Structs (>64-bit)	Spill to Stack Copy	Pointer to caller-allocated temp copy	Pass-by-value (mutation-safe)
Vector / SIMD (<code>__m128</code>)	Spill to Aligned Stack	Pointer to 16/32-byte aligned temp	Pass-by-value

Zult –

Compilation & Execution Pipeline

1. **Static Analysis:** Compiler evaluates variadic arguments at call site, matching inputs against static rules.
2. **Descriptor Generation:** Compiler bakes type enum array into .rodata (e.g., [TYPE_INT32, TYPE_STR, TYPE_STRUCT]).
3. **Caller Frame Setup:**
 - Calculates required payload slots ($N \times 8$ bytes).
 - Allocates isolated stack copies for any large structs or SIMD types to preserve value isolation.
4. **Stack Packing:** Writes direct values and temporary pointers sequentially into contiguous 64-bit slots.
5. **Callee Execution:**
 - Callee reads `payload[i]` via simple $O(1)$ array offset math: `base_address + (i * 8)`.
 - Evaluates `types[i]` at compile-time (if inlined) or via fast jump table (if opaque function).
6. **Zero-Cost Forwarding:** Callee can forward both array slice headers (types and payload) down call chains without stack repacking.

Comparative Trade-off Analysis

- **vs. C (`va_list`):** Eliminates memory corruption risks and manual format string tracking while maintaining low binary size and raw memory speed.
- **vs. C++ (Variadic Templates):** Eliminates executable code bloat caused by generating unique function instances for every type combination.
- **vs. Go / Dynamic Languages (...any):** Eliminates runtime heap allocations, GC tracking overhead, and interface boxing.
- **Primary Trade-off:** Bypasses ABI CPU-register parameter passing (RDI, RSI) for variadic inputs in favor of stack packing to enable contiguous $O(1)$ indexing.